

EMBEDDED SYSTEM AND DESIGN

EXPERIMENT NO. 5

PARKING LOT COUNTER

Name: *Raphael S. Nano*

Course/Section: *CPE160P / BI*

Date of Performance: *September 11-14, 2023*

Date of Submission: *September 15, 2023*

Ma'am Cyrel Manlises
Instructor

DISCUSSION

The number of vehicles in modern times increased drastically due to the evolution and development of technology. The need for containment and spaces for vehicles arose as their number increased. Parking lots and parking spaces became a necessity as people with personal and public vehicles go to various places and temporarily leave it there. Hence, for this experiment, a parking lot counter was constructed with the use of a servo motor, a stepper motor, two ultrasonic distance sensors, and a 4 digit 7-segment display to show the number of available and occupied slots.

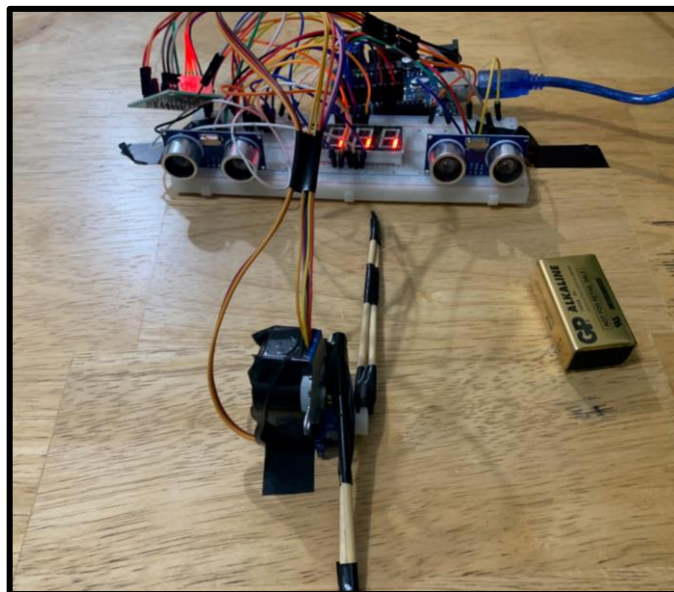


Figure 1. Circuit Setup for Experiment 5

```

5  const int steps = 300;
6  const int trigPin1 = 3;
7  const int echoPin1 = 2;
8  const int trigPin2 = 12;
9  const int echoPin2 = 13;
10 Stepper stepMotor(steps, 13, 11, 12, 10);
11 SevSeg sevseg;
12 Servo servo;
13 int avail = 15;
14 int occupied = 0;
15 int dispNum = 0;
16 void setup() {
17     stepMotor.setSpeed(70);
18     byte numDigits = 4;
19     byte digitPins[] = {9,8,7,6};
20     byte segmentPins[] = {A0,A1,A2,A3,A4,A5,5,LOW};
21     sevseg.begin(COMMON_CATHODE, numDigits,digitPins,
22     segmentPins,false,0,1,0);
23     sevseg.setBrightness(90);
24     servo.attach(4);
25     servo.write(0);
26     pinMode(trigPin1,OUTPUT);
27     pinMode(echoPin1,INPUT);
28     pinMode(trigPin2,OUTPUT);
29     pinMode(echoPin2,INPUT);
30 }

```

Figure 2. Code Setup for Experiment 5

The materials used to construct the parking lot counter, which are shown in Figure 1, are the Arduino UNO, an SG90 Servo motor, a 28BYJ-48 stepper motor, a motor driver module, two HC-sr04 ultrasonic distance sensors, and a 4-digit seven-segment display. The motors and the display all utilized an external library, namely Servo.h, Stepper.h, and SevSeg.h.

```

1  #include <Stepper.h>
2  #include <Servo.h>
3  #include "SevSeg.h"

```

Figure 3. Inclusion of Libraries

The experiment primarily aims to demonstrate the system of a parking lot counter. For the process of the system, there should be two pathways: the entrance and the exit. These pathways should be connected within a single toll-gate. Inevitably, the component that determines the entering and exiting of a car is the ultrasonic distance sensor. By gauging the distance between the sensor and the possible car with respect to the driveway, the programmed motors will move if the cars were detected within the critical distance. The distances were also computed by using the formula $\text{Distance} = (\text{Speed} \times \text{Time}) / 2$, where speed is the sound's speed and time it reaches the object. Specifically, the entering pathway uses 14 as the value of the distance while the exiting path uses 10 as the distance. As shown in Figure 2, the function `distance1()` is for the computation of the distance of the sensor for the entrance, and `distance2()` is for the exit.

```

91 int distance1()
92 {
93     digitalWrite(trigPin1,LOW);
94     delayMicroseconds(2);
95     digitalWrite(trigPin1,HIGH);
96     delayMicroseconds(10);
97     digitalWrite(trigPin1,LOW);
98
99     int duration = pulseIn(echoPin1,HIGH);
100    int distance = duration*0.034/2;
101    return distance;
102 }
103
104 int distance2()
105 {
106     digitalWrite(trigPin2,LOW);
107     delayMicroseconds(2);
108     digitalWrite(trigPin2,HIGH);
109     delayMicroseconds(10);
110     digitalWrite(trigPin2,LOW);
111
112     int duration = pulseIn(echoPin2,HIGH);
113     int distance = duration*0.034/2;
114     return distance;
115 }

```

Figure 3. distance1() and distance2() functions

Moreover, the motors should go up every time the sensor detects an object. The stepper motor simply rotates clockwise for the object to go up, which is shown in the stepUp() function, else counter clockwise to go down, the stepDown(). The servo motor goes to 180 degrees to go up, which is the motorUp() function, and back to 0 to go down, the motorDown(). Moreover, sticks were connected to the ends of the motor so as to imitate a barrier in a toll-gate.

```

46 void stepUp()
47 {
48     stepMotor.step(steps);
49     stepMotor.step(steps);
50     stepMotor.step(steps);
51     delay(500);
52     sevseg.refreshDisplay();
53 }
54 void stepDown()
55 {
56     stepMotor.step(-steps);
57     stepMotor.step(-steps/2);
58     delay(500);
59     sevseg.refreshDisplay();
60 }

```

Figure 4. stepUp() and stepDown() functions

```

62 void motorUp()
63 {
64     sevseg.setNumber(updateSlots());
65     sevseg.refreshDisplay();
66     for (int angle=0; angle<90;angle++)
67     {
68         servo.write(angle);
69         delayMicroseconds(2);
70         sevseg.refreshDisplay();
71     }
72     sevseg.refreshDisplay();
73 }
74
75 void motorDown()
76 {
77     sevseg.setNumber(updateSlots());
78     sevseg.refreshDisplay();
79     for (int angle=90; angle>0;angle--)
80     {
81         servo.write(angle);
82         delayMicroseconds(2);
83         sevseg.refreshDisplay();
84     }
85     sevseg.refreshDisplay();
86 }

```

Figure 5. motorUp() and motorDown() functions

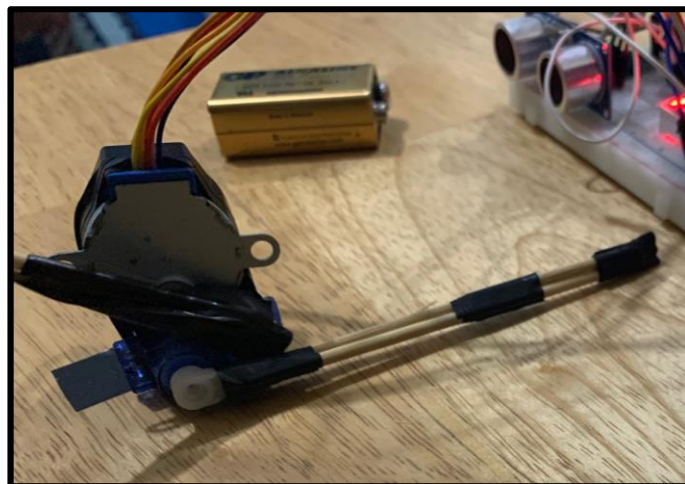


Figure 6. Stepper Motor and Servo Motor Gates

Lastly, the number of slots should be updated as well, depending on the status of cars. The first two digits of the seven-segment display are the occupied slots, while the last two digits are the available slots.

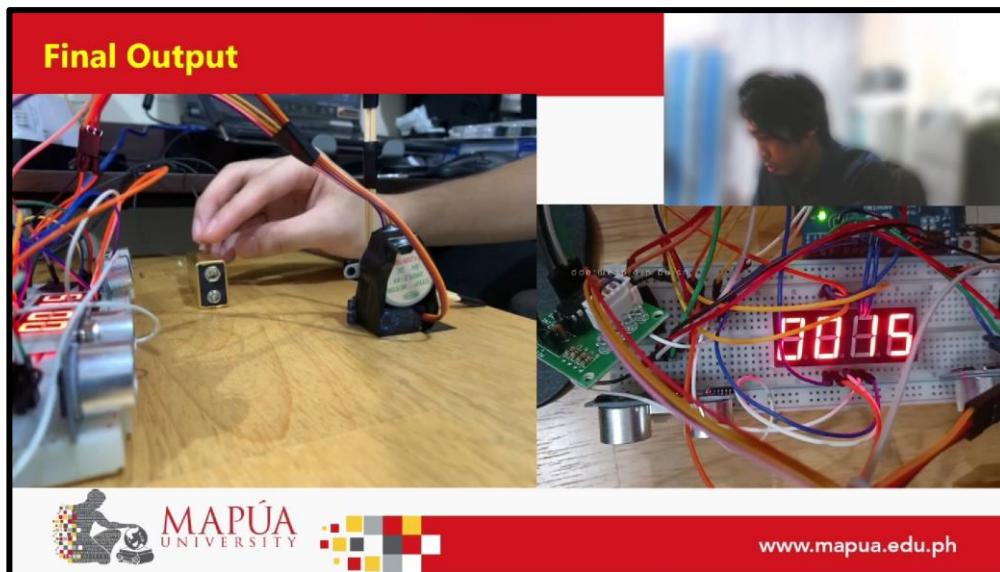


Figure 7.1. Zero (0) occupied slots, Fifteen (15) available slots

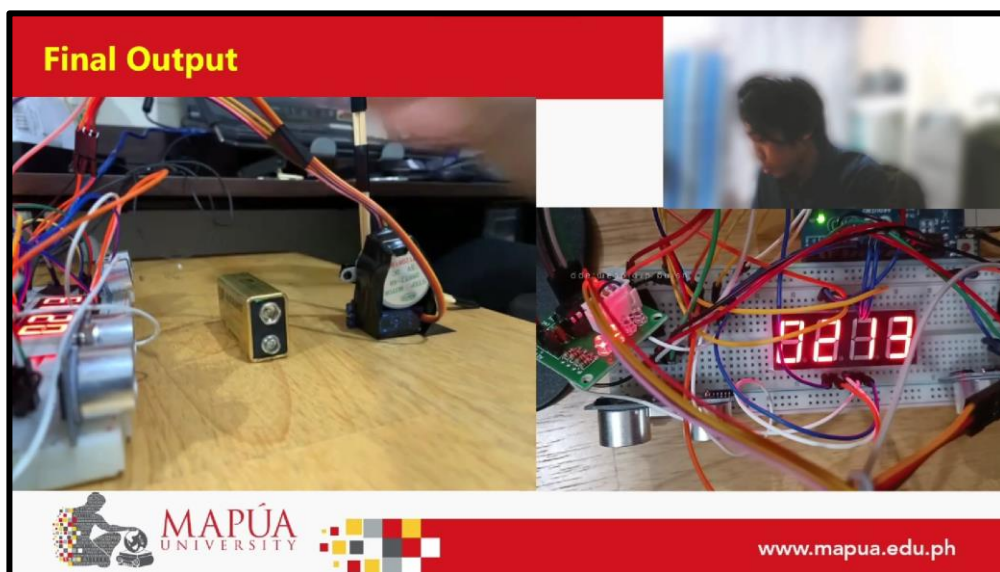


Figure 7.2. Two (2) occupied slots, Thirteen (13) available slots

So, if a car enters, the occupied slots increase while the available slots decrease. On the other hand, if a car exits, the occupied slots decrease while the available slots increase. By using a universal variable, the occupied and the available, for the slots, the manipulation of the values became easier.

```
33  int updateSlots()  
34  {  
35      if (occupied<0)  
36      {  
37          occupied=0;  
38      }  
39      if (avail<0)  
40      {  
41          avail=0;  
42      }  
43      dispNum = (occupied*100)+avail;  
44      return dispNum;  
45  }
```

Figure 8. updateSlots() function

All of the functions mentioned above were combined together in the main loop, but it is again divided into two: a while loop for the entrance ultrasonic distance sensor and a while loop for the exit. The program first checks if the slots are full, and if it is, the program simply continues until the slots are updated by the updateSlots() function when a car has either exited or entered.


```

104 void loop() {
105     sevseg.refreshDisplay();
106     sevseg.setNumber(updateSlots());
107     while (distance1() < 10) {
108         if (occupied == 0) {
109             continue;
110         } else {
111             avail++;
112             occupied--;
113             sevseg.refreshDisplay();
114             motorUp();
115             int startms = millis();
116             int currentms = millis();
117             while (currentms - startms < 5000) {
118                 currentms = millis();
119                 sevseg.setNumber(updateSlots());
120                 sevseg.refreshDisplay();
121             }
122             startms = currentms;
123             sevseg.refreshDisplay();
124             motorDown();
125         }
126     }

```

Figure 9.1. Main loop for ultrasonic distance sensor for entrance

```

128     while (distance2() < 14) {
129         if (avail == 0) {
130             continue;
131         } else {
132             avail = avail - 1;
133             occupied++;
134             sevseg.refreshDisplay();
135             stepUp();
136             int startms = millis();
137             int currentms = millis();
138             while (currentms - startms < 3000) {
139                 currentms = millis();
140                 sevseg.setNumber(updateSlots());
141                 sevseg.refreshDisplay();
142             }
143             startms = currentms;
144             sevseg.refreshDisplay();
145             stepDown();
146         }
147     }
148 }

```

Figure 9.2. Main loop for ultrasonic distance sensor for exit

CONCLUSION

In conclusion, for the first objective, the parking lot counter successfully works wherein if a car enters, the number of occupied slots increase and the number of the available slots decrease in the seven-segment display, and the stepper motor raises the gate, and if a car exits the occupied slots decrease, the available slots increase, and the servo motor raises the gate. For the second objective, as the ultrasonic distance sensors sense an object within critical distance, the value of the count is being updated and checks how many entered and exited as aforementioned. Simultaneously, the motors rotate to raise the gate after sensing an object, which achieves the third objective. The series

connections, pin configurations, and triggering of sensors prove the application of microprocessors, logic circuits, and switching theory. Hence, with the use of various variables, functions, and calling of such functions to develop an embedded system, the parking lot counter, in a simple handshaking.